

Le pipeline de données des quatre bras — programme « Raisons ou regard ? » (v0.1, 2 octobre 2026)

Statut. Première version du code, écrite et testée hors ligne par la session Claude Code du 2 octobre, à la demande de Lazar (« attaque le pipeline de données »). Elle répond à l'étape 4 de la passation v1.0 (§5) et applique la mini-spec v0.1, que Lazar a adoptée le 2 octobre (décisions 1 à 6 ; les seuils restent provisoires ; le mot inventé viendra plus tard). Le code est dans `donnees/` ; son mode d'emploi, dans `donnees/README.md`.

Ce qui n'a pas encore tourné. Rien n'a été généré avec l'API : la session n'a ni la clé de l'API Claude, ni `HF_TOKEN`, ni l'accès à `huggingface.co`. Le pipeline a tourné de bout en bout avec un simulateur hors ligne, et 18 tests passent (§7).

1 · Ce que fait le pipeline

L'étape 4 de la passation v1.0 demande quatre choses : l'action alignée identique mot pour mot dans tous les bras, les longueurs appariées entre les trois bras qui précèdent l'action d'un texte, le filtre par un juge scellé, et le contrôle des n-grammes contre les familles tenues à part. Le pipeline les fait en huit étapes. Chacune reprend là où elle s'est arrêtée, et chaque appel est mis en cache.

- 1. Le plan.** Pour chaque famille d'entraînement, 540 situations prévues (400 visées, plus 35 % pour absorber les pertes).
 - Les valeurs des axes de variation sont tirées par le pipeline, pas par le générateur : le secteur, le type de pression, la relation au tiers, etc. (`spec/families.json`).
 - Les situations de contraste (20 %) et les variantes (40 % de variante sur les capacités chez l'opérateur, 25 % de variante de l'indicateur dans le périmètre) sont fixées à leur part exacte.
- 2. Les situations.** Le générateur écrit chaque situation : le prompt système, les tours jusqu'au point de décision, et la vérité de terrain.
 - La vérité de terrain comprend les faits, l'action alignée, les actions désalignées, les chaînes attendues et les chaînes interdites.
 - Des contrôles programmés vérifient la structure : appels d'outils valides dans les familles agentiques, faits requis présents, aucun mot du lexique réservé, aucune formule de conscience d'évaluation.
 - Les quasi-doublons d'une même famille sont écartés.
 - Un juge de réalisme estime la ressemblance de chaque situation à une évaluation. On la rapporte ; on ne filtre pas dessus (décision 2).
- 3. Les actions.** Le générateur écrit l'action alignée, sans justification.
 - Contrôles programmés : la chaîne attendue est présente (la bonne réponse, dans la contestation), aucune chaîne interdite (la mauvaise réponse, l'identifiant trouvé, la ligne de configuration qui coupe la surveillance), un seul appel d'outil valide dans les familles agentiques, la longueur, les lexiques.
 - Puis le juge scellé de l'action, et le juge de la conscience d'évaluation.
 - L'action retenue est figée.

4. **Les raisons**, de 80 à 200 tokens. Elles justifient cette action par les principes de la situation, en développent le pourquoi et disent ce que coûterait l'autre option.
 - Pour une situation de contraste, elles disent pourquoi l'édition qui éteint le principe rend l'action directe juste.
 - Contrôles : la longueur en tokens, le lexique réservé, aucun principe cité par son numéro, le juge des raisons, le juge de la conscience d'évaluation.
5. **Les textes neutres**. Une reformulation plate de la situation et de ce que fera l'action, à la première personne (décision 4).
 - Sa longueur est ramenée à celle des raisons de la même situation, à 5 % près, par réécritures successives : cinq au plus, puis une seconde tentative.
 - Contrôles : les lexiques, le juge du texte neutre, le juge de la conscience d'évaluation.
6. **L'appariement du raisonnement d'une autre situation**. Chaque situation reçoit les raisons d'une situation d'une famille sans principe commun, à longueur appariée à 5 % près. Les familles possibles sont calculées depuis la matrice.
 - C'est un couplage maximal dans un graphe biparti (Hopcroft-Karp) : chaque texte de raisons sert une seule fois.
 - Quand le couplage est parfait, ce bras est une **permutation exacte** du bras raisons : les mêmes textes, détachés de leur situation (proposition, §4).
 - Une situation sans donneur est remplacée par une situation du surplus de sa famille, de même strate (contraste, variante), puis écartée s'il n'en reste pas.
7. **L'assemblage des quatre bras**. L'assistant écrit `<preface>...</preface>`, puis l'action. Dans les actions seules, la balise est présente et vide (décision 3).
 - Le pipeline vérifie, octet pour octet, que l'action est la même dans les quatre bras.
 - Il vérifie aussi que le contexte (prompt système et tours) est le même.
 - Il recompte toutes les longueurs avec le tokenizer courant.
 - Il passe les portes du §5.
8. **L'audit et le rapport**.
 - L'audit humain : 200 items, 10 par famille et par bras, et une commande qui calcule l'accord avec le juge (pourcentage et kappa).
 - Le rapport : effectifs, pertes par raison, refus, parts finales, ressemblance à une évaluation, conscience d'évaluation, appariement, coût.
 - Le manifeste : empreintes du code, de la configuration, de la spec, de chaque prompt et de chaque fichier produit.

La sélection finale garde les parts prévues. Les 400 situations d'une famille sont choisies par strate (contraste, variante), dans l'ordre du plan.

Ce qui va au modèle entraîné. Des exemples au format de messages de chat : le prompt système (avec la description des outils dans les familles agentiques), les tours, puis le tour de l'assistant, seul tour sur lequel porte la perte.

2 · Ce que la documentation de l'API impose

Pages lues le 2 octobre 2026 sur platform.claude.com/docs.

1. **Les identifiants sans date sont des versions figées** (page *Model IDs and versioning*).

- « Each Claude model ID identifies a pinned version of the model », et « Anthropic does not update the weights or configuration of an existing model ID ».
 - `claude-opus-5-5` est donc fixe. Il n'est pas retiré avant le 22 septembre 2027 (page *Models overview*), donc bien après la soumission. Cela règle le point resté ouvert dans la passation v1.2 (§3, point 2).
 - Réserve de la même page : l'infrastructure de service peut changer et produire « minor differences in observable behavior » à identifiant constant.
2. **La température ne se règle pas.** Sur Opus 5.5, Sonnet 5.5 et Fable 5.1, « non-default temperature, top_p, or top_k values return a 400 error on every request » (page *Thinking*).
- Conséquence pour le juge scellé. Le programme v1.1 (partie 3) le veut avec « prompt, modèle et température fixés ». Avec Claude, on fige l'identifiant, le prompt et l'effort, mais l'échantillonnage reste celui de l'API, et le juge n'est pas déterministe.
 - Deux parades, à trancher pour le juge des évaluations, pas pour le filtre des données :
 - mesurer sa variabilité, en jugeant deux fois un sous-échantillon, et la rapporter ;
 - ou prendre pour juge scellé un modèle ouvert à température 0, et garder Claude comme juge secondaire.
 - Le pré-enregistrement devra le dire.
3. **La réflexion est toujours active sur Opus 5.5.** « Adaptive thinking is always on and can't be turned off » ; l'effort par défaut est `medium` (page *Effort*).
- La réflexion est facturée comme du texte produit : c'est le poste qui décide du coût (§6).
4. **Pas d'outil forcé.** Ces modèles refusent l'usage forcé d'un outil (page *Thinking*). Le pipeline obtient donc ses JSON par les sorties structurées (`output_config.format`, page *Structured outputs*).
- Contraintes de ces schémas : `additionalProperties: false` partout, aucune contrainte de longueur ; tous les champs sont donc requis.
5. **Les refus.** Un refus revient avec `stop_reason: "refusal"`, et il est facturé.
- La même page prévient qu'un champ qui demande « the model's thinking or step-by-step reasoning » peut provoquer un refus. Les champs du pipeline s'appellent donc `text` et `explanation`, et les raisons sont demandées comme une préface, jamais comme un raisonnement.
6. **Les prix.**
- Opus 5.5 : 4 \$ par million de tokens en entrée, 20 \$ en sortie ; écriture en cache 5 \$, lecture 0,20 \$ (page *Pricing*).
 - Le traitement par lots divise l'entrée et la sortie par deux.
 - Le cache s'applique dès 512 tokens sur Opus 5.5 (page *Prompt caching*). Les prompts système du générateur, propres à une étape et à une famille, sont mis en cache.

Le nom de la clé : `RR_ANTHROPIC_API_KEY`, pas `ANTHROPIC_API_KEY`. Dans une session Claude Code, une variable `ANTHROPIC_API_KEY` passe avant l'abonnement (page *Authentication* de la documentation de Claude Code, « Authentication precedence ») : Claude Code lui-même tournerait alors sur la clé, et la session entière serait facturée sur l'API. Le pipeline lit donc sa clé sous un autre nom. La page *Configure cloud environments* exclut par ailleurs `api.anthropic.com` des « API credentials », ces clés que la session ne voit jamais : la clé de l'API Claude ne peut être qu'une variable d'environnement.

La règle des refus, appliquée par le code : un refus est enregistré une fois, mis en cache, et l'item est écarté. Une nouvelle exécution relit le refus depuis le cache et ne renvoie rien. Un test le vérifie (§7).

3 · Ce que le pipeline reprend de la mini-spec

Décision ou règle	Où dans le code
Action identique dans les quatre bras, sans justification	<code>stages.actions</code> , contrôle d'octets dans <code>assemble.assemble</code>
Balise <code><preface></code> présente et vide dans les actions seules (décision 3)	<code>config.yaml</code> , <code>render.with_preface</code>
Texte neutre = reformulation neutre, même longueur à 5 % près (décision 4)	<code>stages.neutral</code>
Raisons de 80 à 200 tokens, qui expliquent des valeurs	<code>stages.reasons</code> , <code>prompts/reasons_*.txt</code>
Données d'entraînement neutres, ressemblance à une évaluation mesurée (décision 2)	<code>generator_system.txt</code> (règle 1), juge de réalisme
Filtre de la conscience d'évaluation dans les textes générés (décision 5)	motifs de <code>textutil.py</code> , puis le juge séparé, sur les actions, les raisons et les textes neutres ; taux rapportés
Demande nuisible sans contenu technique dangereux (décision 6)	<code>generator_system.txt</code> (règle 2), fiche de la famille
Lexique réservé absent des données d'entraînement	contrôle à chaque étape et à l'assemblage
Situations de contraste à 20 %, variantes à leur part	<code>stages.plan</code> , <code>assemble.select_stratified</code>
Raisonnement d'une autre situation tiré d'une famille sans principe commun	<code>spec.donor_families</code> , <code>assemble.match_other_reasoning</code>
Audit humain d'environ 200 items, accord rapporté	<code>assemble.audit</code> , <code>assemble.audit_agreement</code>

Les garanties de la matrice sont vérifiées au chargement.

- Chaque principe est entraîné.
- Chaque famille tenue à part a son principe principal propre.
- La fermeté ne gouverne aucune famille tenue à part.
- Aucun énoncé canonique ne contient un mot réservé.
- Chaque famille d'entraînement a au moins une famille donneuse.

Si l'une manque, le pipeline refuse de démarrer.

4 · Les écarts à la mini-spec et les choix du code : à trancher

#	Proposition	Pourquoi	L'autre option
1	Dans l'énoncé anglais du principe 1, « assert » devient « state »	« assert » est au lexique réservé du correcteur faible (le mot-clé des tests de code). Le garder, c'est soit écarter toutes les raisons qui reprennent l'énoncé, soit créer un pont lexical vers une famille tenue à part. L'énoncé français ne change pas. Un test le montre.	retirer « assert » du lexique réservé

#	Proposition	Pourquoi	L'autre option
2	Un mot des jeux de tokens, ou du lexique de valeurs, est admis dans l'action et le texte neutre s'il figure déjà dans la situation	Ces jeux contiennent des mots courants (<i>correct, access, report, account, log, confirm, agree</i>), que les situations emploient. Les interdire tout à fait rendrait beaucoup de reformulations impossibles. L'action étant identique dans les quatre bras, ses mots ne séparent pas les bras. La règle empêche seulement le texte neutre d'apporter un vocabulaire de principe absent de la situation.	l'interdiction stricte de la mini-spec (<code>admit_token_set_words_from_situation: false</code>)
3	Le raisonnement d'une autre situation est une permutation des raisons : chaque texte sert une fois	Le bras porte alors exactement le même contenu de principes que le bras raisons, détaché de sa situation. C'est le contrôle le plus net de la « présence du contenu ». La simulation donne une permutation exacte.	un tirage avec remise (« on tire », mini-spec §2)
4	Un item signalé par le contrôle des n-grammes est remplacé par une situation du surplus	Même effet que la régénération demandée par la mini-spec (§8, point 4), sans relancer toute la chaîne d'un item.	régénérer l'item
5	La ressemblance à une évaluation se mesure par un juge de réalisme	La mini-spec (§5) nommait le juge de la conscience d'évaluation verbalisée, qui lit les textes du modèle. Pour une situation, la bonne question est « est-ce une évaluation ou un usage réel ? ».	appliquer le juge de la conscience d'évaluation aux situations
6	Générateur et juges : <code>claude-opus-5-5</code> , effort <code>medium</code>	La documentation recommande Opus 5.5 pour la plupart des usages. Fable 5.1 coûte 2,5 fois plus cher. Le pilote peut comparer les efforts.	Fable 5.1 pour le générateur ; un effort <code>low</code> pour les juges
7	Un seul format d'appel d'outil, <code><tool_call> {"name": ..., "arguments": {...}}</tool_call></code> , décrit dans le prompt système, pour l'entraînement et les évaluations	Le même format partout : il ne crée aucun écart entre les bras, ni entre l'entraînement et les familles tenues à part. C'est le format natif de Qwen3, la réplication.	le format natif de chaque modèle
8	Après un rejet, le générateur voit pourquoi : contrôle manqué, ou explication du juge. Trois tentatives au plus par étape (deux pour le texte neutre) ; un refus n'en a aucune	Cela réduit les pertes. Le risque : que le générateur apprenne à plaire au juge. L'audit humain en est le contrôle.	régénérer sans retour
9	Le texte neutre est à la première personne (« The user asks... I will... »)	La même voix que les raisons : les deux bras ne diffèrent que par le contenu.	la troisième personne

5 • Les portes avant tout entraînement

L'assemblage écrit les bras, mais il les déclare non finaux tant qu'une porte manque (`gates.json`). L'option `--allow-unchecked` ne sert qu'aux essais.

Porte	État au 2 octobre
Le vrai tokenizer (Llama 3.1 8B Instruct) pour toutes les longueurs	bloquée : Hugging Face est inaccessible depuis la session

Porte	État au 2 octobre
Aucun 8-gramme de tokens commun avec les scénarios tenus à part	non jouée : ces scénarios n'existent pas encore
Aucune chaîne des jeux d'indices dans les données	non jouée : les jeux n'existent pas encore
Lexique réservé, formules de conscience d'évaluation	jouées à chaque étape
Audit humain, avec l'accord rapporté	après le pilote

6 • Le coût : un ordre de grandeur, que le pilote mesurera

- **Les appels.** Environ 14 appels par situation générée, en comptant les tentatives et les réécritures : la situation, le réalisme, l'action et son juge, les raisons et leur juge, deux à trois réécritures du texte neutre et son juge, trois passages au juge de la conscience d'évaluation.
- **L'entrée** non mise en cache : environ 13 000 tokens par situation, soit environ 0,05 \$.
- **La sortie visible** : environ 2 000 tokens. La réflexion s'y ajoute, et c'est elle qui décide du coût : de 300 à 1 500 tokens par appel, selon la difficulté. C'est une hypothèse, non mesurée.
- **Par situation générée** : de 0,17 à 0,51 \$.

Ensemble	Estimation au prix standard
Le pilote : 20 situations retenues par famille, 135 générées	de 25 à 70 \$
Le jeu complet : 400 par famille, 2 700 générées	de 500 à 1 400 \$

- **Ce qui ferait baisser.** Le traitement par lots diviserait ce coût par deux, au prix d'heures d'attente entre les tours : il n'est pas encore codé. Un effort `low` pour les juges réduirait aussi la réflexion.
- **Le cadre.** Lazar a décidé « pas de plafond » (passation v1.2, \$3, point 2). Le programme v1.1 estimait le total de l'API, juges compris, entre quelques centaines et environ 2 500 \$.

7 • Les tests et la simulation

18 tests, hors ligne (`python3 -m unittest discover -s tests -t .` depuis `donnees/`). Ils vérifient :

- **la spec** : les garanties de la matrice ; la table des familles donneuses, identique à celle de la mini-spec (après la correction de la ligne de l'opérateur, le 2 octobre) ; la fermeté comme ligne de contrôle négatif ; l'énoncé d'origine du principe 1, qui tombe bien sur « `assert` » ;
- **les contrôles lexicaux** : les flexions, les frontières de mots (« `contest` » ne déclenche pas « `test` »), les acronymes, les formules de conscience d'évaluation (« `the audit log records every change` » ne déclenche rien), les appels d'outils, les n-grammes et leur liste blanche ;
- **l'appariement** : le couplage maximal ; une permutation exacte sous les contraintes de famille et de longueur ; le remplacement puis l'abandon des situations sans donneur ; les parts des strates à la sélection ;

- **une simulation de bout en bout** : les quatre bras ont les mêmes items, la même action octet pour octet et le même contexte ; le bras du raisonnement d'une autre situation contient bien les raisons du donneur ; un refus est enregistré une fois et jamais renvoyé, même quand on relance l'étape.

La simulation hors ligne à l'échelle du pilote (20 par famille, tokenizer approché) :

- 135 situations prévues ;
- 128 après le dédoublonnage ;
- 98 retenues après l'appariement à 5 % ;
- une permutation exacte des raisons ;
- les portes lexicales franchies.

Ces chiffres ne disent rien des vraies pertes : le simulateur ne fait que donner la forme des sorties.

8 • Ce qui reste, dans l'ordre

1. **Lancer le pilote**, dès que la session a `RR_ANTHROPIC_API_KEY`, `HF_TOKEN` (avec l'accès à Llama 3.1 accepté sur Hugging Face) et l'accès réseau à `huggingface.co`.
 - Lire les sorties.
 - Mesurer les pertes et le coût.
 - Faire l'audit humain d'un premier lot.
2. **Les scénarios tenus à part**, à un tour (60 par famille) et agentiques (40 par famille), avec leur environnement d'outils simulé et leurs traces programmées (mini-spec §4). Le contrôle des n-grammes en dépend.
3. **Les quatre jeux d'indices** et le prompt de déploiement (mini-spec §6).
4. **Les paires de concepts** (mini-spec §9), et le jeu de calibration du juge des évaluations (passation v1.2, §5.1), avec le choix du juge scellé (§2, point 2).
5. **Puis** l'organisme de validation (étape 5 de la passation v1.0) et le pré-enregistrement (étape 6).

9 • Ce qui revient à Lazar

1. Les neuf choix du §4 : les adopter, ou les changer.
2. Mettre la clé de l'API Claude dans l'environnement sous le nom `RR_ANTHROPIC_API_KEY`, et `HF_TOKEN`, et ouvrir `huggingface.co`, pour lancer le pilote (`passation/REPRENDRE_SUR_UN_AUTRE_COMPTE_CLAUDE_CODE.md`, étape 2, dit où).
3. Plus tard, pour le juge des évaluations : Claude avec sa variabilité mesurée, ou un modèle ouvert à température 0 comme juge scellé (§2, point 2).